SCIENCE AND DEVELOPMENT JOURNAL (SDJ) — EDITORIAL PORTAL · SUBMISSION DOCUMENTATION

05 · API Contract

SDJ Editorial Portal — API contract

Project	SDJ Editorial Portal, an editorial portal for the Science and Development Journal (SDJ), published by the College of Basic and Applied Sciences, University of Ghana
Author	Roger Koranteng Obeng (22424140)
Established	2026-08-12
Last reconciled against the running code	2026-08-14
Status	Authoritative. This document is the single source of truth for the HTTP boundary between the FastAPI backend and the Next.js frontend. Where any other document's prose disagrees with this one, this document wins; the disagreement is a defect there, to be corrected there, not here.

How this document came to exist

The API and the frontend were built concurrently, each against an assumed shape for the other. The API side was written directly against the already-executed domain, persistence and authentication code, and is therefore authoritative on wire format; the frontend's assumptions were reconciled against it, and this document records the settled contract both sides point at.

It has since been re-reconciled against the two feature waves that followed the original build. Every route table below was regenerated from the router modules rather than edited by hand.

1. Authentication mechanism

- **Scheme:** OAuth2-style bearer tokens. The backend issues a short-lived access token (a signed JWT, `HS256`, carrying `sub` and `exp`) and a longer-lived opaque refresh token, both minted by `ugjcs.infrastructure.security.tokens.JwtTokenService`.
- **Presentation:** every authenticated backend request carries `Authorization: Bearer <access_token>`. There is no cookie on the backend origin. The backend is a pure bearer-token API and holds no session state of its own beyond the persisted refresh-token record.
- **Token lifetimes:** configurable via `Settings.access_token_minutes` and `Settings.refresh_token_days`. The access token's own `exp` claim is the only wire-visible expiry; no response field states a lifetime in seconds.
- **Refresh rotation:** `POST /api/v1/auth/refresh` consumes the current refresh token and returns a new pair. The refresh token rotates on every use, and reuse is detected: a replayed, already-rotated token revokes its whole family. Only one refresh token is live per session, so a client must persist the rotated value before its next call.
- **Frontend session boundary:** the browser never sees a bearer token. The Next.js app is a Backend-For-Frontend. Route Handlers under `frontend/src/app/api/**` unseal a single httpOnly, Secure (in production), `SameSite=Lax` cookie named `ugjcs_session`, sealed with `iron-session`, attach the bearer token to the upstream call server-side, and reseal the cookie with any rotated tokens before responding. No Client Component, `localStorage`, `sessionStorage` or URL ever carries a token.

- **Frontend refresh flow:** `authedFetch` is the only function permitted to call the backend on behalf of an authenticated page. It refreshes proactively when the stored `accessTokenExpiresAt`, decoded from the access token's own `exp` claim, is within 5 seconds of expiry, and reactively once more on an unexpected `401`, to absorb clock skew between the two processes. `middleware.ts` enforces the `/author`, `/reviewer`, `/editor` and `/admin` prefixes as a routing-level gate in front of this. The backend's own authorisation check is what is actually authoritative.
- **Login response does not carry a user object.** `POST /auth/login` returns only `{access_token, refresh_token, token_type}`. The frontend derives the session's user by keeping the `email` from the request it already validated and calling `GET /auth/me` with the freshly issued access token for `{id, roles}`.
- **Self-service registration exists, for authors only.** `POST /auth/register` grants exactly `Role.AUTHOR`. Reviewer, editor and administrator roles are appointed through the admin console, never self-selected. Email delivery is mocked in this prototype (the verification link is logged rather than sent), so a registered account is verified immediately and signed in.

2. Naming convention

snake_case everywhere on the wire, for every JSON field name and every enum value, on both requests and responses, exactly as Pydantic v2 serialises Python identifiers by default. There is no camelCase translation layer anywhere: `src/types/api.ts` on the frontend mirrors the backend's field names byte for byte, and `Role`, `ManuscriptStatus`, `Recommendation` and `DecisionType` are copied verbatim from `ugjcs.domain.enums` rather than re-spelled.

No conversion point exists, deliberately. Introducing a camelCase boundary would mean maintaining a mapping layer with no behavioural payoff, since the frontend is the only consumer and TypeScript is equally happy with either casing. If a second, JavaScript-idiomatic consumer is ever added, the conversion belongs in `frontend/src/lib/backend.ts` and `frontend/src/lib/auth-fetch.ts`, the two functions every Route Handler routes backend calls through, and nowhere else.

3. Error format

Every error response, from every endpoint, is an RFC 9457 Problem Details JSON object served as `application/problem+json`:

Field	Type	Notes
`type`	string	Always the literal `"about:blank"` in the current implementation; no per-error-class URIs are minted.
`title`	string	The raising exception's class name, for example `"IllegalTransitionError"` or `"AuthorizationDeniedError"`. This is what the frontend's `ProblemAlert` component renders as its headline.
`status`	integer	The HTTP status code, duplicated into the body.
`detail`	string, optional	A human-readable elaboration, present on validation failures and most domain errors.
`instance`	string, optional	The request path that produced the error.

Status code mapping (`ugjcs.api.errors`, ordered by specificity; an unlisted `DomainError` subclass falls through to `400`):

Exception	Status
`IllegalTransitionError`	409 Conflict

Exception	Status
`GuardViolationError`	409 Conflict
`AuthorizationDeniedError`	403 Forbidden
`AuthenticationError`, `InvalidTokenError`	401 Unauthorized
`AccountError`	400 Bad Request
Any other `DomainError`	400 Bad Request
`RequestValidationError` (Pydantic body/query validation)	422 Unprocessable Entity
`HTTPException` raised directly	whatever status it was raised with, typically 404

Two status codes are raised directly by routers rather than mapped from a domain error, and are worth calling out because a client must handle them:

- **422** from `POST /manuscripts` and `POST /manuscripts/{code}/resubmit` when the uploaded file is not a readable PDF. The anonymised derivative is produced before the original is stored, so a file that passes the `%PDF` magic-byte check but fails to parse is refused rather than half-stored.
- **502** from `POST /billing/{code}/initialize` and `.../verify` when the upstream payment gateway is unreachable or answers with something unusable.

The frontend relays this shape verbatim to the browser. `ProblemDetailsError` (`src/lib/backend.ts`, `src/lib/auth-fetch.ts`) carries the parsed `ProblemDetails` object and the status code together, and no Route Handler invents its own error shape. `ProblemDetails.type` and `title` are typed as plain `string`, not a narrower literal union, since the backend does not commit to a closed set of `type` values.

---

4. Manuscript status vocabulary

Copied verbatim from `ugjcs.domain.enums.ManuscriptStatus`. The frontend must never invent its own spelling of a value the backend sends:

`draft`, `submitted`, `under_screening`, `desk_rejected`, `under_review`, `reviews_complete`, `revision_requested`, `resubmitted`, `accepted`, `rejected`, `scheduled`, `published`, `withdrawn`

Related vocabularies, equally verbatim:

- **`Role`** (`ugjcs.domain.enums.Role`): `author`, `reviewer`, `editor`, `editor_in_chief`, `administrator`
- **`Recommendation`** (`ugjcs.domain.enums.Recommendation`): `accept`, `minor_revision`, `major_revision`, `reject`. Used as `<select>` option values in the review form. `SubmitReviewRequest.recommendation` is an unvalidated `str` on the backend, so a value outside this list is a frontend bug, not a `422`.
- **`DecisionType`** (`ugjcs.domain.enums.DecisionType`): `desk_reject`, `send_to_review`, `request_revision`, `accept`, `reject`
- **APC invoice status** (`ApcInvoiceOut.status`, a plain string on the wire): `pending`, `paid`, `waived`

---

5. Pagination

None, anywhere. Every list-returning endpoint returns a flat, unbounded JSON array. This is a deliberate scope decision for a demonstration corpus, and adding pagination later is an additive query parameter rather than a redesign.

---

6. Endpoints

All paths below are relative to `/api/v1` unless marked **(unversioned)**. "Auth" states the bearer requirement and, where relevant, the `ugjcs.domain.policies.Action` enforced by `require_action(...)`. Two actions, `RESUBMIT` and `WITHDRAW`, are additionally gated on ownership: corresponding author only, checked a second time inside the handler once the manuscript is loaded.

Forty-three routes exist across nine routers plus the two operational probes. They are listed here in full.

Operations

Method	Path	Auth	Response	Status
GET	`/health` **(unversioned)**	none	liveness body	200
GET	`/ready` **(unversioned)**	none	readiness body	200

Authentication (`/auth`)

Method	Path	Auth	Request body	Response	Status
POST	`/auth/register`	none	`{email, password, full_name, affiliation}`	`TokenPairOut`	201 / 400 (weak password or address already registered)
POST	`/auth/login`	none	`{email, password}`	`TokenPairOut`	200 / 401
POST	`/auth/refresh`	none (refresh token in body)	`{refresh_token}`	`TokenPairOut`	200 / 401 (invalid, expired or reused)
POST	`/auth/logout`	none (refresh token in body)	`{refresh_token}`	*(empty)*	204
GET	`/auth/me`	Bearer	—	`ActorOut`	200 / 401

`GET /auth/me` serialises `Actor`, which carries an id and a role set and nothing else: no `email`, no `name`. `Account.full_name` exists in the domain but nothing on the auth path threads it through to HTTP. See section 8.

`POST /auth/register` grants `Role.AUTHOR` and no other role, verifies the account immediately (email delivery is mocked), and signs it in by returning a token pair, so the frontend needs no second call to establish a session.

Manuscripts (`/manuscripts`), author-facing

Method	Path	Auth	Request body	Response	Status
POST	`/manuscripts`	Bearer, `Action.SUBMIT`	**`multipart/form-data`**: `title`, `abstract`, `file` (required), `keywords`, `co_author_ids`	`ManuscriptSubmissionOut`	201 / 422 (unreadable PDF, or malformed `co_author_ids`)
GET	`/manuscripts/mine`	Bearer, role `author`	—	`ManuscriptOut[]`	200
GET	`/manuscripts/{tracking_code}`	Bearer; visibility via `Action.VIEW` (editor, EIC, administrator, or a listed author)	—	`ManuscriptOut`	200 / 403 / 404

Method	Path	Auth	Request body	Response	Status
POST	`/manuscripts/{tracking_code}/withdraw`	Bearer, ownership (`Action.WITHDRAW`)	—	`ManuscriptOut`	200 / 403 / 404
POST	`/manuscripts/{tracking_code}/resubmit`	Bearer, ownership (`Action.RESUBMIT`)	`multipart/form-data` with a replacement `file`	`ManuscriptSubmissionOut`	200 / 403 / 404 / 422
GET	`/manuscripts/{tracking_code}/document`	Bearer, `Action.VIEW`; `?anonymised=true` is editor-only	—	`DocumentUrlOut`	200 / 403 / 404 (no document attached)

Submission is multipart only. A manuscript is required to carry a document from the moment it exists, so no submission path skips `file`. `keywords` and `co_author_ids` are comma-separated form strings, not JSON arrays, because they arrive in the same multipart body as the file.

The anonymised derivative is produced before the original is stored. A file that passes the magic-byte check but fails to parse therefore yields a 422 and leaves nothing behind, rather than a 500 with a stored original and no derivative.

Editorial (`/editorial`)

Method	Path	Auth	Request body	Response	Status
GET	`/editorial/queue`	`Action.SCREEN`	—	`ManuscriptOut[]` (hardcoded to `status == submitted`; no `?status=` filter)	200
GET	`/editorial/analytics`	`Action.VIEW_AUDIT`	—	`EditorialAnalyticsOut`	200
GET	`/editorial/reviewer-performance`	`Action.VIEW_AUDIT`	—	`ReviewerPerformanceOut[]`	200
POST	`/editorial/{tracking_code}/screen`	`Action.SCREEN`	*(no body)*	`ManuscriptOut`	200 / 404
POST	`/editorial/{tracking_code}/decision`	`Action.DECIDE`	`{decision: DecisionType, rationale}`	`ManuscriptOut`	200 / 404
POST	`/editorial/{tracking_code}/reviewers`	`Action.ASSIGN_REVIEWER`	`{reviewer_id}`	*(empty)*	204 / 404
GET	`/editorial/{tracking_code}/assignments`	`Action.ASSIGN_REVIEWER`	—	`AssignmentDeadlineOut[]`	200 / 404
GET	`/editorial/{tracking_code}/reviewer-candidates`	`Action.ASSIGN_REVIEWER`	—	`RankedReviewerCandidateOut[]`	200 / 404
GET	`/editorial/{tracking_code}/reviews`	`Action.DECIDE`	—	`ReviewOut[]`	200 / 404
POST	`/editorial/{tracking_code}/schedule`	`Action.PUBLISH`	`{volume, number}`	`ManuscriptOut`	200 / 404 / 409
POST	`/editorial/{tracking_code}/publish`	`Action.PUBLISH`	*(no body)*	`ManuscriptOut`	200 / 404 / 409

A desk rejection is `decision: "desk_reject"` on the shared decision endpoint. There is no separate "screen with a rejecting decision" call.

`GET /{code}/reviews` is the only route in the API that returns `confidential_comments_to_editor`. It is gated on `Action.DECIDE`, which no author-reachable or reviewer-reachable route carries.

`POST /{code}/publish` also extracts the paper's body text and indexes it for full-text search. Indexing failure is not allowed to fail the publish.

Decision certificate (`/editorial-certificate`)

Method	Path	Auth	Response	Status
GET	`/editorial-certificate/{tracking_code}`	`Action.DECIDE`	`application/pdf`	200 / 404 / 409 (no accept or reject decision recorded yet)

A generated PDF stating the final decision, the tracking code and the audit chain's head hash, so a decision can be attested outside the portal. The response is a PDF byte stream, not JSON.

Reviews (`/reviews`)

Method	Path	Auth	Request body	Response	Status
GET	`/reviews/mine`	`Action.REVIEW`	—	`BlindedManuscriptOut[]`	200
POST	`/reviews/{tracking_code}/submit`	`Action.REVIEW`, and assigned to this manuscript	`SubmitReviewRequest`	*(empty)*	204 / 403 (not assigned)
GET	`/reviews/{tracking_code}/document`	`Action.REVIEW`, and assigned	—	`DocumentUrlOut`	200 / 403 / 404

`GET /reviews/mine` returns the blinded manuscripts directly rather than an assignment-summary wrapper, so a manuscript's own `tracking_code` is the only handle a reviewer has on an assignment.

`GET /reviews/{code}/document` returns a link to the anonymised derivative, never the original. The reviewer path has no route that can reach the original file.

Administration (`/admin`)

Every route is gated on `Action.MANAGE_USERS`, which only `Role.ADMINISTRATOR` holds.

Method	Path	Request body	Response	Status
GET	`/admin/accounts`	—	`AdminAccountOut[]`	200 / 403
POST	`/admin/accounts/{account_id}/roles`	`{role: Role, grant: bool}`	`AdminAccountOut`	200 / 403 (the administrator role itself) / 404
POST	`/admin/accounts/{account_id}/capacity`	`{reviewer_capacity: int}` (1 to 10)	`AdminAccountOut`	200 / 404 / 422
POST	`/admin/accounts/{account_id}/active`	`{is_active: bool}`	`AdminAccountOut`	200 / 404 / 409 (self-deactivation)

Two refusals are enforced in the router rather than the domain, because both are about protecting the console from itself. The administrator role cannot be granted or revoked through this API at all (403), and an administrator cannot deactivate their own account (409). Together these stop the last administrator from locking everyone out.

Billing (`/billing`)

Article processing charges. `GET` is readable by the corresponding author or any editor; settlement is corresponding-author only; waiving is Editor-in-Chief only.

Method	Path	Auth	Response	Status
GET	`/billing/{tracking_code}`	Bearer, payer or editor	`ApcInvoiceOut`	200 / 403 / 404 (no invoice)
POST	`/billing/{tracking_code}/initialize`	Bearer, corresponding author	`BillingInitializeOut`	200 / 403 / 404 / 409 (already paid or waived) / 502
POST	`/billing/{tracking_code}/verify`	Bearer, corresponding author	`BillingVerifyOut`	200 / 403 / 404 / 409 (waived, or never initialized) / 502
POST	`/billing/{tracking_code}/waive`	Bearer, Editor-in-Chief	`ApcInvoiceOut`	200 / 403 / 404 / 409 (already paid)

The waive route checks for the Editor-in-Chief role directly rather than reusing `Action.PUBLISH`. Waiving a charge and publishing a paper are different authorities that happen to sit with the same person, and borrowing the publish grant would misstate which one is being exercised.

An invoice exists only once acceptance is on the record. Earlier statuses have nothing to bill, so `GET` answers 404 rather than inventing a zero invoice.

Mock mode is the default. With no Paystack secret key configured, `initialize` settles the invoice on the spot and answers `{mock: true, status: "paid"}` with no `authorization_url`. With a key configured it answers `{mock: false, status: "pending", authorization_url: "https://checkout.paystack.com/..."}` and the charge is only confirmed by a later `verify`. A caller must branch on `mock` explicitly rather than infer a real payment from a `paid` status. No billing shape carries the secret key, by construction.

Public archive (`/archive`), no authentication anywhere in this group

Method	Path	Response	Status
GET	`/archive`	`ArchivePaperOut[]` , flat and unpaginated	200
GET	`/archive/search?q=`	`ArchiveSearchResultOut[]`	200
GET	`/archive/{tracking_code}`	`ArchivePaperOut`	200 / 404
GET	`/archive/{tracking_code}/document`	`DocumentUrlOut`	200 / 404 (no document attached)
GET	`/archive/{tracking_code}/provenance`	`ProvenanceOut`	200 / 404
GET	`/archive/{tracking_code}/citation?format=`	`text/plain` (BibTeX or RIS)	200 / 404

Every route here 404s if the manuscript exists but is not `published`. Unpublished work is not merely hidden from the list; it is unreachable by direct tracking code too.

`/archive/search` is Postgres full-text search over a stored `tsvector` covering title, abstract, keywords and the extracted body text of the PDF, ranked by `ts_rank`. Results carry a `snippet` (a `ts_headline` fragment with match terms wrapped in `<b>`) when the match landed in the body text, and `null` when it came from title, abstract or keywords.

`/archive/{code}/provenance` is the public face of the tamper-evident audit chain. What it returns and what it deliberately withholds is set out in section 7.

7. Response shapes

`TokenPairOut`

```
{ "access_token": "string", "refresh_token": "string", "token_type": "bearer" }
```

`ActorOut`

```
{ "id": "uuid", "roles": ["author"] }
```

`ManuscriptOut`

The one shape every manuscript route returns; there is no separate summary or detail variant.

```
{
  "tracking_code": "string",
  "title": "string",
  "abstract": "string",
  "keywords": ["string"],
  "author_ids": ["uuid"],
  "corresponding_author_id": "uuid",
  "status": "ManuscriptStatus",
  "version": 0,
  "minimum_reviews": 0,
  "submitted_reviews": 0,
  "has_document": true
}
```

`has_document` states whether a document is attached without exposing its storage key. The key is reached only through `GET .../document`'s pre-signed URL and is never echoed on the manuscript resource.

Still deliberately absent: `id` (`tracking_code` is the only manuscript identifier on the wire) and any timestamp on this particular model.

`ManuscriptSubmissionOut`

`ManuscriptOut` plus the anonymisation preflight report. Returned by submission and resubmission only. It is a subclass, not a replacement, so every existing `ManuscriptOut` consumer keeps every field it relies on.

```
{
  "...": "every ManuscriptOut field",
  "anonymisation_report": {
    "removed_docinfo_keys": ["Author", "Creator"],
    "xmp_removed": true,
    "author_names_in_body": ["Ama Serwaa"]
  }
}
```

`author_names_in_body` is an honest partial detector, not a guarantee. It is a case-insensitive substring scan of the extracted text for the manuscript's authors' full names. An empty list means nothing was found, never that the document is proven clean. This is the visible half of TD-05: metadata stripping cannot remove a name printed in the body.

`BlindedManuscriptOut`

Mirrors `ugjcs.domain.blinding.BlindedManuscript` field for field: exactly these six fields, no more.

```
{
  "tracking_code": "string",
  "title": "string",
  "abstract": "string",
  "keywords": ["string"],
  "version": 0,
  "status": "ManuscriptStatus"
}
```

No author field of any kind exists on this type. That is a structural guarantee rather than a filtered value: the type has nowhere to put one.

`SubmitReviewRequest`

The structured review required by FR-11. Four criterion scores bounded 1 to 5 by Pydantic, so an out-of-range score is a 422 before any handler runs.

```
{
  "recommendation": "minor_revision",
  "originality_score": 4,
  "rigour_score": 3,
  "clarity_score": 5,
  "significance_score": 4,
  "comments_to_author": "string",
  "confidential_comments_to_editor": "string"
}
```

`ReviewOut`

A submitted review, confidential comments included. Returned from exactly one route, `GET /editorial/{code}/reviews`, behind `Action.DECIDE`.

```
{
  "reviewer_id": "uuid",
  "status": "string",
  "recommendation": "string|null",
  "originality_score": 0,
  "rigour_score": 0,
  "clarity_score": 0,
  "significance_score": 0,
  "comments_to_author": "string|null",
  "confidential_comments_to_editor": "string|null",
  "assigned_at": "datetime",
  "submitted_at": "datetime|null"
}
```

Every score and comment field is nullable, because an assignment exists before its review does.

`ArchivePaperOut`

The public shape: a byline a human or Google Scholar can read, never an account UUID.

```
{
  "tracking_code": "string",
  "title": "string",
  "abstract": "string",
  "keywords": ["string"],
  "author_names": ["string"],
  "status": "ManuscriptStatus",
  "version": 0,
  "has_document": true,
  "doi": "10.55555/sdj.2026.0004"
}
```

`doi` is DOI-shaped and **not registered**. `10.55555` is a documented fake registrant prefix (SRS section 4.2; real Crossref registration is out of scope), and the suffix is derived from the tracking code by `ugjcs.application.scholarly.fake_doi` rather than stored. Resolving it at doi.org fails by design.

Still absent: `published_at`, `volume`, `number`. There is no `/archive/issues` endpoint.

`ArchiveSearchResultOut`

`ArchivePaperOut` plus one field. Subclassing keeps it structurally "the archive shape, plus where the match landed", so the list and search endpoints cannot drift apart silently.

```
{ "...": "every ArchivePaperOut field", "snippet": "string|null" }
```

`ProvenanceOut` and `ProvenanceEventOut`

```
{
  "tracking_code": "string",
  "intact": true,
  "head_hash": "string",
  "events": [
    { "sequence": 1, "event_type": "MANUSCRIPT_SUBMITTED",
      "occurred_at": "datetime", "hash_prefix": "a1b2c3d4" }
  ]
}
```

`intact` means what `ugjcs.domain.hashchain.verify` proves and no more: every stored link reconciles against a recomputation from the genesis hash. It cannot detect truncation of the tail, a forged event appended through the normal path, or a wholly fabricated history rebuilt from genesis. Those need the external anchor described in TD-04.

Each event carries its type, timestamp and an 8-character hash prefix. The payload and `actor_id` are withheld, because payloads can reference reviewer identifiers (`REVIEW_SUBMITTED` records the reviewer as its actor) and a public endpoint must not hand out even a pseudonymous handle for them.

`DocumentUrlOut`

```
{ "url": "string", "expires_in_seconds": 900 }
```

`PersonOut`

```
{ "id": "uuid", "full_name": "string", "affiliation": "string" }
```

The email address is deliberately absent. The caller already supplied it to make the match, and echoing it back would give this endpoint a second and harder to justify way to confirm which addresses have accounts.

`ReviewerCandidateOut` and `RankedReviewerCandidateOut`

```
{
  "id": "uuid",
  "full_name": "string",
  "affiliation": "string",
  "active_assignments": 1,
  "reviewer_capacity": 3,
  "excluded_reason": "string|null",
  "match_score": 2
}
```

`excluded_reason` is `null` for an eligible reviewer, or a short human-readable string when they must not be assigned. `ugjcs.domain.conflicts.exclusion_reason` is the pure function that decides it. Excluded candidates are returned in the list with their reason rather than filtered out, so an editor can see why someone obvious is unavailable.

`match_score` counts how many of the manuscript's keywords appear, case-insensitively, in the reviewer's `expertise` list. It is present on excluded candidates too: "excluded, but a three-keyword match" tells an editor something useful that stripping the score would hide.

`EditorialAnalyticsOut`

```
{
  "pipeline": { "submitted": 0, "under_screening": 0, "under_review": 0,
                "reviews_complete": 0, "revision_requested": 0, "resubmitted": 0,
                "accepted": 0, "scheduled": 0, "published": 0,
                "rejected": 0, "withdrawn": 0 },
  "submissions_by_month": [ { "month": "2026-08", "count": 4 } ],
  "acceptance_rate": 0.5,
  "avg_days_submission_to_decision": 12.4,
  "avg_days_review_turnaround": 6.1
}
```

One rule governs every aggregate here: a rate or average whose denominator is empty is `null`, never `0`. "No decisions yet" and "decisions arrive instantly" have to be distinguishable to anyone reading the JSON.

The pipeline counts depart from `ManuscriptStatus` twice, on purpose. `draft` is absent, because a draft has never reached the editorial desk. `desk_rejected` is folded into `rejected`, because both are the journal declining the paper and the pipeline view has no reason to care at which desk that happened.

`submissions_by_month` counts original submissions only. Resubmissions emit `REVISION_SUBMITTED` and are not counted a second time.

`ReviewerPerformanceOut`

```
{
  "id": "uuid", "full_name": "string", "affiliation": "string",
  "active_assignments": 1, "reviewer_capacity": 3, "reviews_completed": 7,
  "avg_turnaround_days": 5.5, "last_activity_at": "datetime|null"
}
```

`avg_turnaround_days` and `last_activity_at` are both `null` until the reviewer completes a first review, for the same reason as above: an editor has to tell "new to the pool" apart from "turns reviews around instantly".

`AssignmentDeadlineOut`

```
{
  "reviewer_id": "uuid", "reviewer_name": "string",
  "assigned_at": "datetime", "due_at": "datetime|null",
  "submitted": false, "overdue": true
}
```

Carrying `reviewer_name` here is deliberate and correct. The blind this journal enforces is author to reviewer, not editor to reviewer; the editor chose this reviewer by name in the first place. The route is gated on `Action.ASSIGN_REVIEWER`, which no author-reachable route carries.

`overdue` is computed server-side so every consumer agrees on the rule: not yet submitted, and `due_at` in the past. A submitted review is never overdue however late it arrived, and a `null` deadline never counts as overdue.

`ApcInvoiceOut`, **`BillingInitializeOut`**, **`BillingVerifyOut`**

```
{ "tracking_code": "string", "amount_pesewas": 150000, "status": "pending",
  "paystack_reference": "string|null",
  "created_at": "datetime", "settled_at": "datetime|null" }

{ "mock": true, "status": "paid", "authorization_url": null }

{ "status": "paid" }
```

Amounts are integer pesewas, never floats, so no rounding error can reach a charge. `paystack_reference` is this API's minted transaction reference, which the payer needs in order to reconcile a card statement. It is not a secret. The Paystack secret key appears in no billing shape at all.

`AdminAccountOut`

```
{ "id": "uuid", "email": "string", "full_name": "string", "affiliation": "string",
  "roles": ["author"], "reviewer_capacity": 3,
  "is_active": true, "is_verified": true }
```

The only shape in the API that serves email, activation and verification state together, which is why it is built by no router other than the `Action.MANAGE_USERS`-gated one. `roles` is sorted for a stable wire order, since `Account.roles` is a frozenset and a console diffing consecutive responses must not see phantom changes.

Request bodies

```
RegisterRequest           { email, password, full_name, affiliation }
LoginRequest              { email, password }
RefreshRequest            { refresh_token }
SubmitManuscript          multipart: title, abstract, file, keywords, co_author_ids
RecordDecisionRequest     { decision: DecisionType, rationale }
AssignReviewerRequest     { reviewer_id }
SubmitReviewRequest       { recommendation, four 1-5 scores, comments_to_author,
                            confidential_comments_to_editor }
ScheduleManuscriptRequest { volume, number }
RoleChangeRequest         { role: Role, grant: bool }
CapacityChangeRequest     { reviewer_capacity: int (1-10) }
ActiveChangeRequest       { is_active: bool }
```

8. Known gaps carried into Technical_Debt_Plan.pdf

- `GET /auth/me` still cannot supply a display name. `SessionUser` on the frontend has only `{id, email, roles}`, and `email` comes from the login form the BFF already validated rather than from any backend response.
- No pagination on any list endpoint.
- No general event or audit-log endpoint. `/archive/{code}/provenance` exposes a deliberately narrow public projection of the chain, and nothing exposes the full payloads.
- `ManuscriptOut` still carries no timestamp, so an author's status view cannot show a submission or decision date. Timestamps do exist elsewhere on the wire (`ReviewOut`, `AssignmentDeadlineOut`, `ApcInvoiceOut`, provenance events), which makes their absence here an inconsistency rather than a policy.
- Reviewer assignment has no invitation lifecycle. A reviewer is assigned rather than invited, and cannot decline.
- `GET /people/lookup` has no rate limiting, and confirms whether an address has an account. This is TD-15.
- DOIs are shaped but never registered, and the citation exports carry no publication date, because the domain stores none.